The River and the Canyon — Lean Edition

A Physical Analogy for Transformers, in Brief

E. A. Flores · Apiana AI, Inc. · May 2026

> This is the condensed version of a longer paper. It teaches the whole picture — tokenization through generation — in one fast pass, then states where the analogy breaks. The full version develops each step in more depth and stress-tests every claim against real mechanism. **Read the full paper »**

Most explanations of large language models are either too loose ("a digital brain that thinks") or too dense (a wall of linear algebra). This one takes a third path: it maps the real operations of a transformer onto a single physical picture and holds that picture consistently from raw text to generated word.

The core mapping, in one breath: **the weights are a frozen mountain; the activations are water flowing over it.** Training is the slow carving of the rock. Inference is water finding paths across stone that no longer moves. The governing distinction is **permanence** — grooves carved by training are permanent; the channels water traces on a single pass are not. That one axis is the difference between weights and activations, and the whole analogy hangs on it.

One warning to carry, collected in full at the end: the water *appears* to fall, but there is no force in the picture. "Down" means later in the computation, not lower in energy. Hold "flow" as *computation advancing*, not motion under a pull.

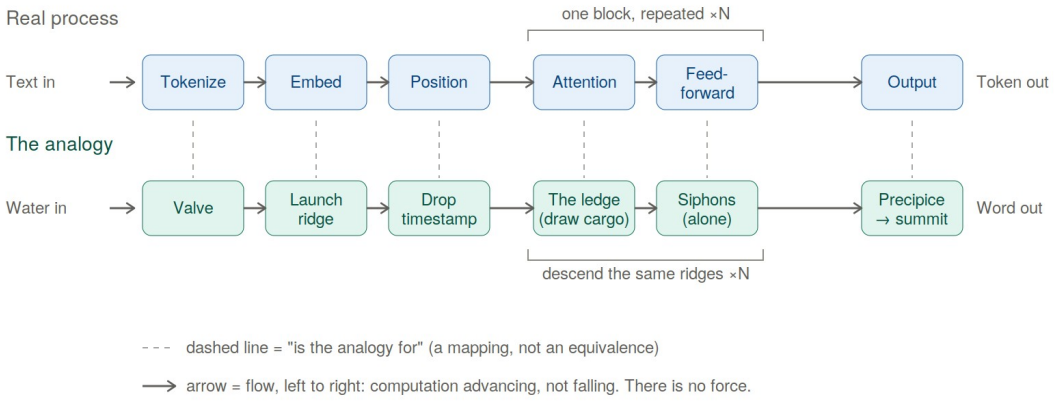

Figure 1. The forward pass, with each real step aligned above its analogy — the real process over the picture, stage by stage. Dashed lines mean "is the analogy for," not "is the same as." The diagram shows one pass, producing one token; generation repeats it — each output token is appended and the whole pass runs again for the next word.

Part I — How the Mountain Works

Tokenization. A separate, mechanical step slices text into sub-word units and converts them to integer IDs from a fixed vocabulary. The model never reads text — only IDs. *In the analogy:* a valve breaks the continuous stream of language into discrete, standardized drops.

Embeddings. Each token ID is looked up in a trained matrix and becomes a dense vector (say, 1024 or 4096 numbers). Training arranges these so words used alike point in similar directions — "king" near "queen" — with no engineer coding the link; meaning is purely geometric. *In the analogy:* each drop type has a fixed launch point on the ridge, and similar drops were sculpted into adjacent positions during construction. Crucially, an ambiguous word like *bank* launches from the same averaged coordinate every time — pulled partway toward *river* and partway toward *money* at once. The launch point is a **prior**; the descent is what resolves it into a specific meaning.

Positional encoding. Attention is order-blind on its own, so position must be injected — either added as an offset before the stack, or (in modern models, via RoPE) folded in by rotating the Query and Key vectors by an angle set by position. Either way, "dog bit man" stays distinct from "man bit dog." *In the analogy:* each drop is stamped with a timecode, or twisted by its position, so otherwise-identical drops strike the rock at distinguishable angles.

The residual stream. Modern layers don't overwrite the token's vector; they *add* to it (x + attention, then x + feed-forward), with a normalization step keeping magnitudes stable. There's one channel per token, running in parallel — and the only operation that lets one token draw on another is attention. *In the analogy:* every drop rides its own deep-cut riverbed — its Central Run — from top to bottom, keeping the same channel the whole way. The drop never swells into a puddle: it stays one fixed-width vector, but its *composition* grows richer as it descends.

Attention is where tokens exchange context, and it's the heart of the machine. Each token projects three vectors through trained matrices: a **Query** (what context am I looking for?), a **Key** (what do I advertise?), and a **Value** (what do I actually hand over — which need not match what I advertise). A token's Query is matched against every Key; the match strengths become proportions (via softmax); and the token's new value is the blend of every Value, weighted by those proportions.

In the analogy: at a ledge, the rock briefly exposes each Central Run to its neighbors. Each drop carries three things — a **question**, a **placard** (what it advertises), and a **cargo** (what it gives). A drop's question is checked against every placard, and match-strength sets what fraction of each cargo pours into its run. In "the boat drifted toward the bank," the word *bank* finds *boat* and *drifted* the strongest match, their cargo pours in, and the drop leaves shifted toward the riverside meaning. Change the upstream words to "the teller counted the cash" and the same starting drop slides into the financial meaning instead. **The rock never moved; the neighbors changed.** Three things stay true: the weights are mixing proportions, not a selection (every neighbor contributes its share at once); the pour is one-directional (a drop drawing from another doesn't alter that other); and the rock gives up no substance of its own — the terrain only supplies the matching, never the cargo.

The key line to keep: grooves carved by training are permanent; the channels attention lays down exist only for that one input and vanish when it's done. *Permanence is the whole distinction.*

Multi-head attention. Rather than one attention over the full width, the space splits into parallel heads, each working in its own slice and appearing to specialize (syntax, reference, sentiment) — though those are interpretive readings, not labels the model assigns. *In the analogy:* the stream splits its perception into several independent senses, which merge back at the ledge's base.

The feed-forward block is attention's counterpart: after tokens share context, each passes *alone* through a small network, transformed in isolation with no cross-token talk. *In the analogy:* having drawn from one another, the drops are forced one at a time through narrow, pressurized siphons, each processed purely on its own properties, then spilled back into its run. Every layer alternates the two: gather and mix, then squeeze each alone.

Stacking. One layer isn't enough; transformers stack dozens to a hundred-plus, each layer's output feeding the next, with a rough division of labor (surface patterns low, syntax middle, abstract meaning high) that's gradient, not tiered. *In the analogy:* the water descends ridge after ridge, its composition shifting at each, until by the final precipice the simple drops have become a sophisticated current reflecting the whole journey.

From water to word. After the last layer, the final-position vector is projected against the vocabulary, producing one raw score (logit) per possible token. Softmax turns those into a probability distribution; a temperature dial controls how sharp it is; a sampling rule then picks one token. That token is appended to the input, and the *entire descent runs again* for the next word. *In the analogy:* below the final lip is a fan of outflow notches, one per word, each cut to a depth set by its score. The water commits to one — but here's the move the downhill image hides: it isn't released into a sea. It's carried back to the summit as the single new drop of the next pass. The river never reaches an ocean; it runs the whole range once per word.

Training vs. Inference

The whole of machine learning hinges on a distinction that looks like two states of matter but is really one material under different amounts of flow.

Training feeds data forward, measures error against a target, and propagates that error backward to nudge every weight (gradient descent). *In the analogy:* this has two movements. First **construction** — at initialization there's no terrain, only randomness, and the early violent phase is where the mountain *comes into being*; this is the one genuinely malleable moment. Then **refinement** — once the terrain stands, continued training is water cutting grooves into *already-solid* rock. This corrects the old "soft clay, then fired" picture: the rock doesn't have to be soft to be carved, it just takes sustained flow. The Grand Canyon wasn't cut while the rock was clay. (One precise fact worth keeping: the water that carves is *language* — the mountain never meets the world, only descriptions of it. The full paper takes up what that does and doesn't imply about understanding.)

Inference is what happens once training ends and the weights lock: a query computes activations, but not one parameter changes. This is why a base model doesn't remember your

conversation after the session clears. *In the analogy:* the flood has stopped. One stream runs once, far below the dosage that carves. The same rock erosion was cutting a moment ago is now simply being *run over*. The illusion of a mind responding to you is purely the attention step — dynamic routing on top of an entirely static substrate.

The nuance: "frozen" means what one pass can't move, not what nothing ever can. Fine-tuning cranks the flow back to carving dosage and reshapes the standing rock — but it's bounded. It can deepen channels and connect existing high ground into new routes; it cannot freely re-raise the range. Not "fired and unchangeable," but "shaped freely only once, and merely carvable ever after."

A Few Production Techniques

- **KV caching.** Generating each new token would mean recomputing attention over all prior tokens; instead, their Keys and Values are cached. *In the analogy:* each earlier drop leaves a temporary wet trace of its Keys and Values at the ledges it already crossed; a new drop reads those traces instead of making every old drop descend again, so only its own fresh splash is computed. The traces are left *by* the water — the rock itself is unchanged.
- **FlashAttention.** A way to compute *exact* attention without ever writing the giant attention matrix to slow memory — it works block-by-block in fast on-chip memory, carrying running totals. *In the analogy:* instead of photographing the whole cliff at once, you study one square foot at a time with blinders on, keeping a running tally in the margin so the final map is exactly as accurate. The speed is all in avoiding slow memory traffic.
- **LoRA.** Instead of re-training billions of weights, freeze them and inject two small trainable matrices alongside. *In the analogy:* LoRA changes the route without recarving the main rock — a light, removable guide at a critical choke point diverts the river without moving an ounce of stone.

Part II — Where the Picture Breaks

A good analogy earns trust by showing where it fails. The discipline is simple: **state each claim with no analogy in the sentence, say what would prove it wrong, and check.** The failures cluster, and the cluster is the real finding.

The deepest break: there is no force. Every page has water falling — and that's the analogy's central error. At inference the model isn't searching, relaxing toward a low-energy state, or being pulled toward the likely answer; it's evaluating a fixed function, layer by layer. There is no driving force at all — not gravity, not least resistance. The terrain doesn't *push* the water; the terrain *is what the water becomes*. "Down" is bookkeeping for "later in the stack." Even "finds the easy path" is the same ghost — the path isn't found, it's computed directly. This is load-bearing for the whole picture, and it will feel natural to forget, which is exactly why it's the most dangerous.

Model merging (a clean break). Two models fine-tuned from the same base can often be averaged into one that keeps both strengths — and the analogy says it's because they "share bedrock." Wrong variable. What actually governs mergeability is compatible parameter *geometry* (whether the solutions sit in the same low-loss basin, accounting for permutation

symmetries), not shared ancestry. The mountain reached for the cause it could *draw* instead of the one that decides the outcome — its most instructive failure.

Quantization (a bend). Shrink a model's numbers from sixteen bits to four and it mostly keeps working — the analogy expects this. But it assumes the coarsening is *uniform*, and it isn't: a small minority of weights carry a disproportionate load, and low-bit methods work precisely by protecting those and crushing the rest. Right phenomenon, wrong texture — smooth where the material is lumpy.

Interpretability (a structural wall). Can you read an existing mountain backward to find where a concept "lives"? Largely no — because of **superposition**: the model packs far more features than it has dimensions, stored as overlapping directions rather than clean axes. So a single direction has no single meaning, and **the picture cannot tell you where a capability lives, because in a superposed system capabilities don't live anywhere you could point.** (The 3D mountain captures the *kind* of crowding faithfully; what it can't show is the *scale* — how much high-dimensional space can hold before interference dominates.)

The near-miss (the one that nearly fooled me). The analogy's central claim is about inference, so I designed an experiment to test it: freeze a model's downstream layers, fine-tune the upper layers until a behavior degrades, then feed the frozen layers their *original* input and see if the behavior returns. A vivid, fundable result — until I wrote the claim with no mountain in it and it collapsed. **Feeding a frozen (and therefore fixed) function the exact input it originally received returns the original output by construction — guaranteed, for any model, any task.** The experiment was a closed loop around an un-falsifiable identity; it was mathematically impossible to fail, and so could teach nothing. What survives is a real experiment: not *whether* restoring the whole input recovers the behavior (it must), but *how little* of it must be restored — which genuinely measures where the damage concentrates. That near-miss, and the rule that caught it, are the reason the full paper exists.

What It All Means

The analogy has teeth in exactly one circumstance: when a phenomenon turns on the line between **permanent and temporary** — between weights and water. It bends or breaks the moment the deciding variable lives elsewhere — in geometry (merging), salience (quantization), hardware (the speed of inference), or optimizer dynamics. Where the governing variable leaves the permanence boundary, the mountain can still point at the right neighborhood, but it cannot name the mechanism — and worse, it will confidently pretend it can.

So the transferable rule, good for any intuition about these systems:

1. **Can you state the claim with no analogy in the sentence?** If not, you don't have a claim — you have an image.
2. **Does it secretly assume uniformity** where the real system is lumpy?
3. **Does the real answer depend on something the picture can't see** — geometry, hardware, optimization?

If the deciding variable lies outside the picture's reach, let it point you to the neighborhood, then set it down. That's the whole result: **not a mountain, but a method for not being fooled by mountains** — including the most seductive one, the one you build yourself.

And the wonder survives all of this. When a model produces brilliant prose, it's tempting to imagine a ghost in the machine. The picture forces the plainer truth: no hidden hand, just a structure shaped by data until it captured the geometry of language. That there is no agent behind the rock is a real result. Whether the structure *understands* what it learned — that, the mountain cannot settle, and doesn't pretend to. The river never truly fell, and the canyon never truly pulled. There was only a fixed shape and water taking it.

One question this leaves open: text is only one kind of water. Code, video, simulation, and action carry different structure — and whether each carves a different geometry is a question this picture raises but does not answer. A companion piece, *What Kind of Water Carves the Mountain?*, takes it up.

This is the lean edition. The full paper develops each step in depth, embeds diagrams, and stress-tests every claim with sources. Written by E. A. Flores, Apiana AI, Inc. Licensed CC BY-NC 4.0.